

— TECHNICAL REPORT

AdamW vs Hybrid Muon for MiniGPT Pretraining

A controlled single-GPU benchmark across TinyStories and
FineWeb-Edu token caches.

Lean ML Visual Labs

Version 1.0 · 2026.07

Local L40S benchmark report

| Contents

01	Executive Summary	03
02	Question and Optimizer Setup	04
03	Model, Data, and Controls	06
04	Results	09
05	Interpretation and Limits	13
06	Appendix	15

01 · EXECUTIVE SUMMARY

Executive Summary

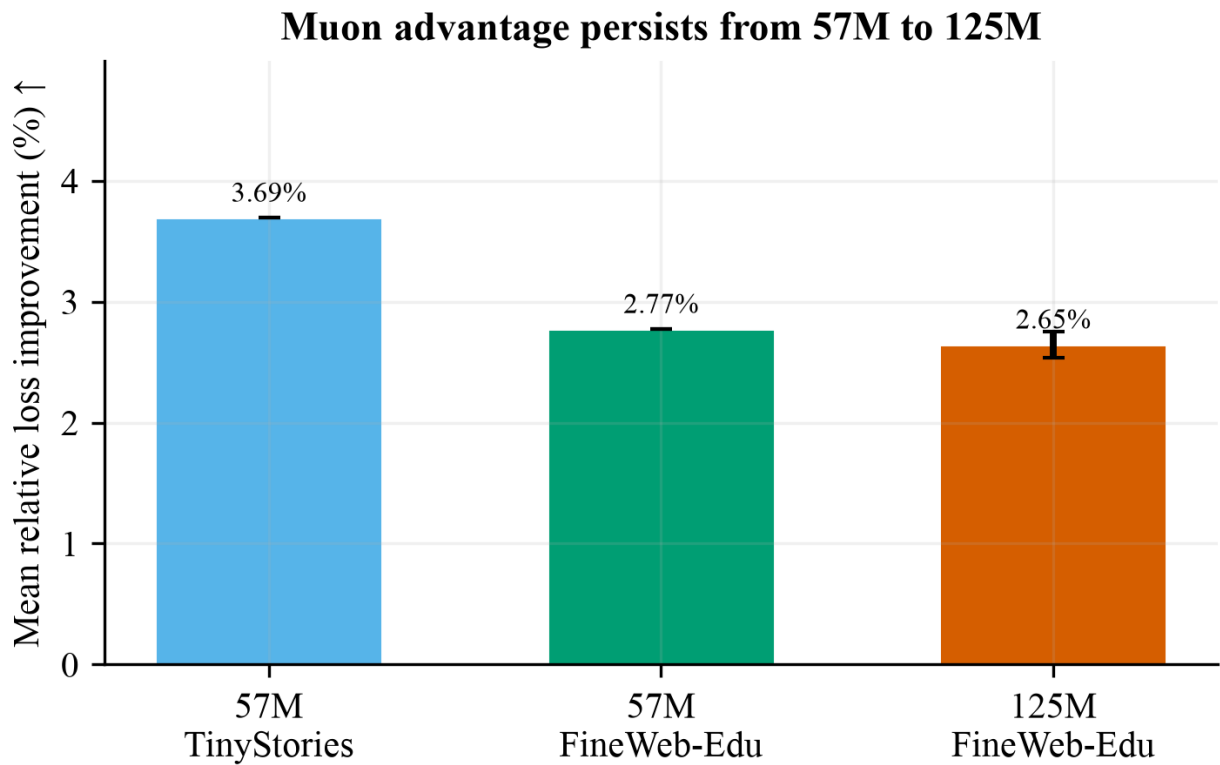
We compared [AdamW](#) against [Hybrid Muon](#) for decoder-only MiniGPT pretraining. Across a narrow short-story corpus and a broader educational web-text cache, Hybrid Muon reached lower validation loss at matched model, seed, batch, and token-budget settings.

Key Takeaways

- On the 57M MiniGPT model, Hybrid Muon improved mean best validation loss by [3.69%](#) on TinyStories and [2.77%](#) on the 500M-token FineWeb-Edu cache over 3 seed-matched 4000-step runs.
- On the 125M MiniGPT model, Hybrid Muon improved mean best validation loss by [2.65%](#) across 3 FineWeb-Edu seeds.
- AdamW was faster in raw throughput. The result supports a narrower claim: Hybrid Muon delivered better validation loss per training token in this controlled setup.

MAIN CLAIM

In our single-GPU MiniGPT experiments, Hybrid Muon consistently reached lower validation loss than AdamW under fixed architecture, data cache, seed, batch size, and token budget. This is not a claim that Muon universally dominates AdamW.



Summary of mean relative validation-loss improvement across the completed 57M and 125M experiments.

02 · QUESTION

Question and Optimizer Setup

The benchmark isolates one question: if the model, data, seed, batch size, and token budget are held fixed, does changing the optimizer from AdamW to Hybrid Muon change validation loss?

What “Hybrid Muon” means

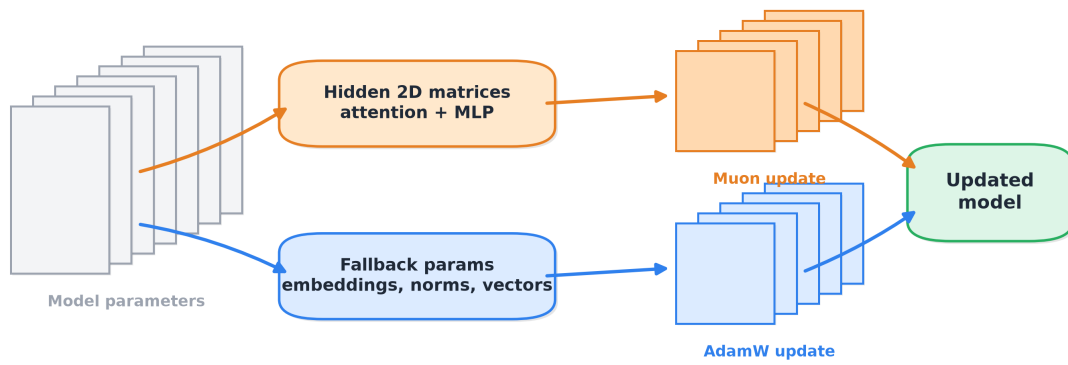
Hybrid Muon is not Muon applied to every parameter. It is a split optimizer: Muon handles selected hidden two-dimensional matrix weights, while AdamW handles embeddings, normalization parameters, vectors, scalars, and fallback parameters.

Parameter group	Optimizer	Why
Attention Q, K, V, O projection matrices	Muon	Large hidden 2D matrices where matrix-aware updates are meaningful.
MLP up and down projection matrices	Muon	Large 2D matrices mapping $768 \rightarrow 3072 \rightarrow 768$.
Token embeddings and position embeddings	AdamW	Embedding tables are treated as fallback parameters in this implementation.
LayerNorm weights, biases, vectors, scalars	AdamW	Non-matrix or small parameters do not use Muon in this hybrid recipe.

This split is why the report uses the term Hybrid Muon. The comparison is AdamW for all eligible parameters versus a mixed Muon-plus-AdamW optimizer.

Hybrid Muon optimizer split

Same model parameters, two update rules: matrix-aware Muon for hidden 2D weights, AdamW for fallback parameters



Muon is not replacing AdamW everywhere. The point of the paper implementation is the split.

This makes the benchmark fair: same model, same data, same seed, same token budget — only optimizer logic changes.

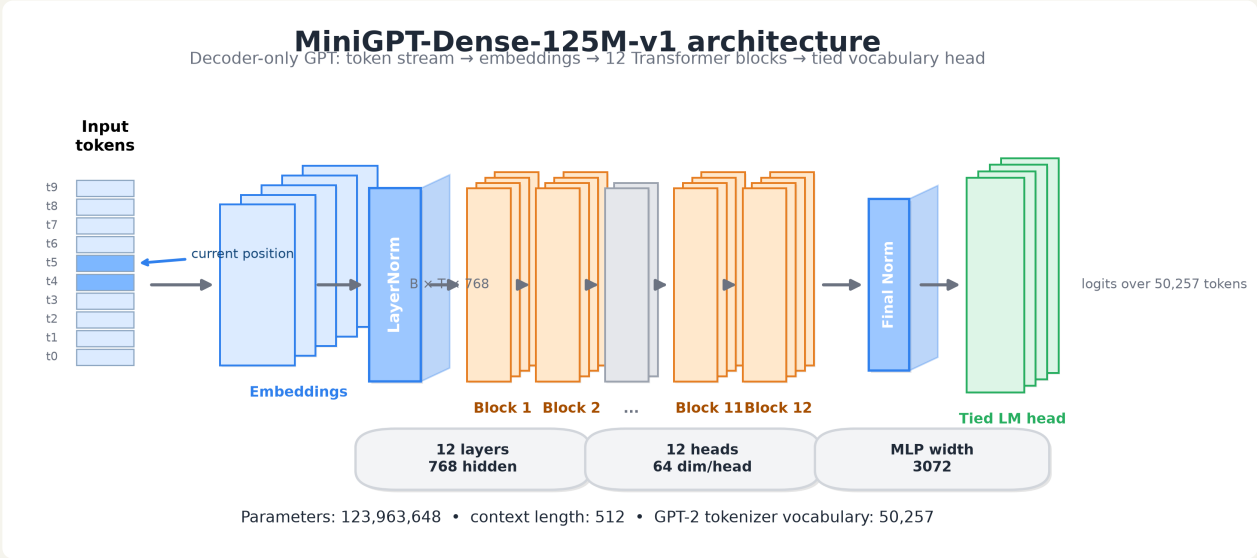
Hybrid Muon optimizer split used in the experiments: Muon for hidden matrices, AdamW for fallback parameters.

Model, Data, and Controls

The experiment uses decoder-only GPT-style models implemented in PyTorch. The main scale result uses MiniGPT-Dense-125M-v1, a GPT-2-small-style configuration with context length 512.

MiniGPT-Dense-125M-v1 architecture

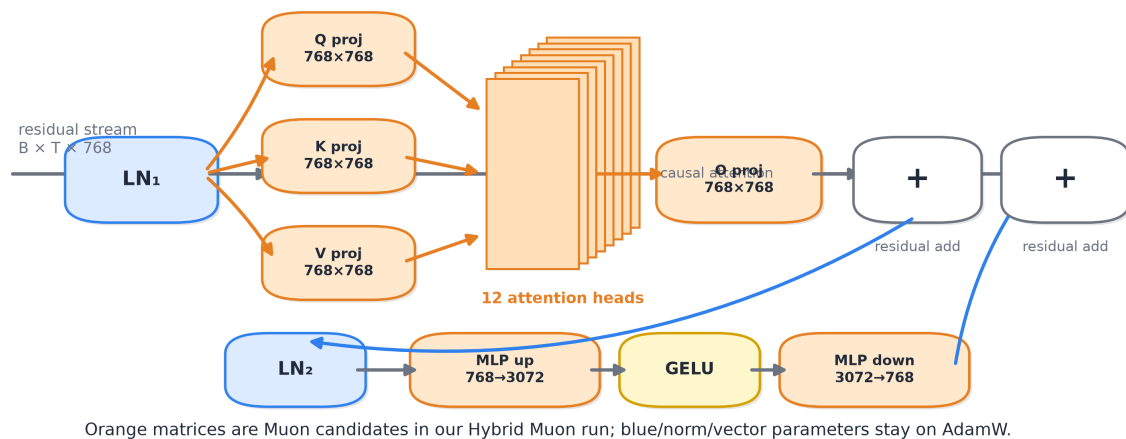
Dimension	Value
Parameters	123,963,648
Layers	12
Hidden size	768
Attention heads	12
Head dimension	64
MLP width	3072
Context length	512
Tokenizer vocabulary	50,257



Code-generated 2.5D schematic of MiniGPT-Dense-125M-v1.

One Transformer block, exploded

The repeated unit inside MiniGPT: attention path + MLP path wrapped by residual connections



Exploded view of one Transformer block: attention path, MLP path, and residual additions.

Dataset caches

The report should state cache size and consumed-token budget separately. Cache size describes how much tokenized data was available. Tokens seen describes how much each training run actually consumed.

Dataset cache	Training tokens in cache	Validation tokens in cache	Role in report
TinyStories	473,992,236	4,765,918	Narrow short-story corpus, used in 57M runs.
FineWeb-Edu sample-10BT extract	500,000,000	5,000,000	Broader educational web-text cache, used in 57M and 125M runs.

Training controls

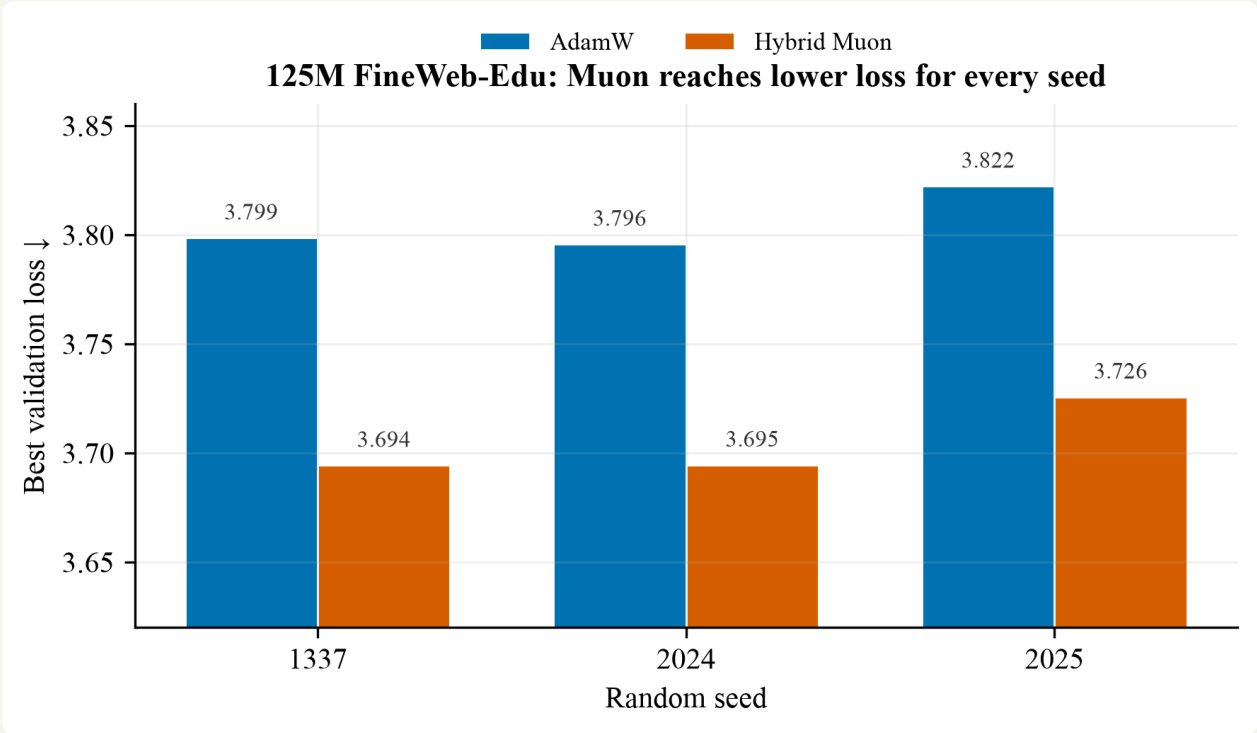
Control	Value
Effective batch tokens	65,536
4000-step tokens per run	262,144,000
Validation tokens per evaluation	262,144
125M benchmark seeds	1337, 2024, 2025
Precision	bf16
Hardware	Single NVIDIA L40S

TINYSTORIES USAGE

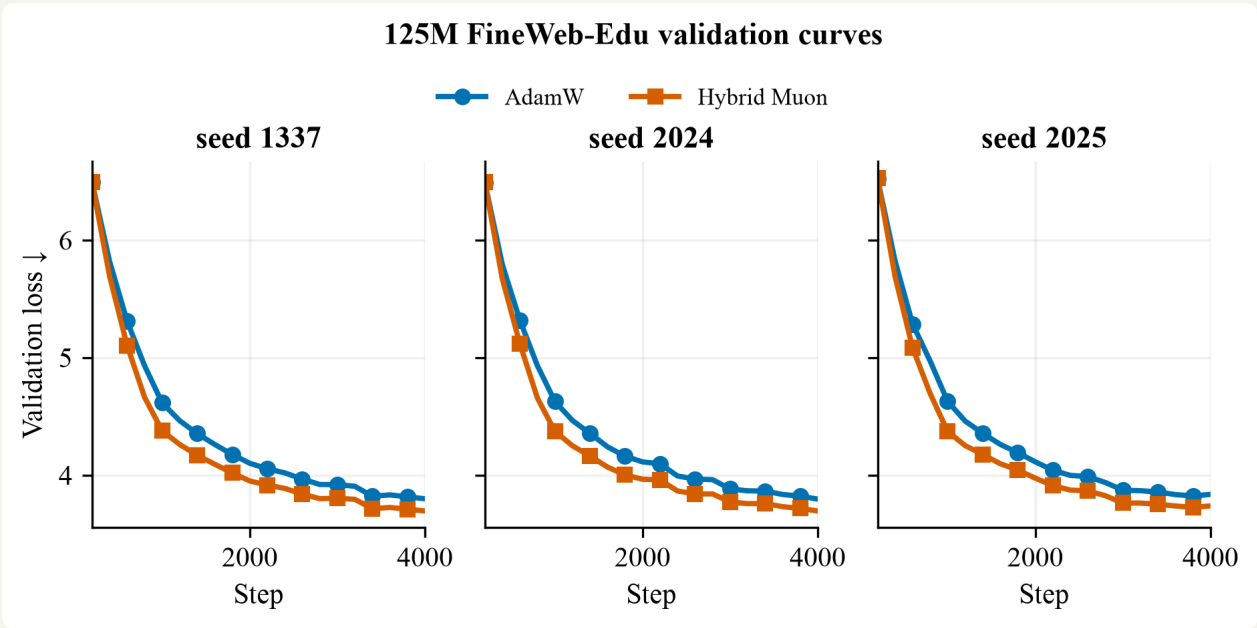
Each 4000-step TinyStories run consumed 262.14M tokens, about 55.3% of the 473.99M-token training cache, assuming no full wraparound.

Results

The strongest result is the 125M, 3-seed FineWeb-Edu benchmark. Hybrid Muon reached lower validation loss in every seed-matched pair.



125M FineWeb-Edu best validation loss by seed. Lower is better.

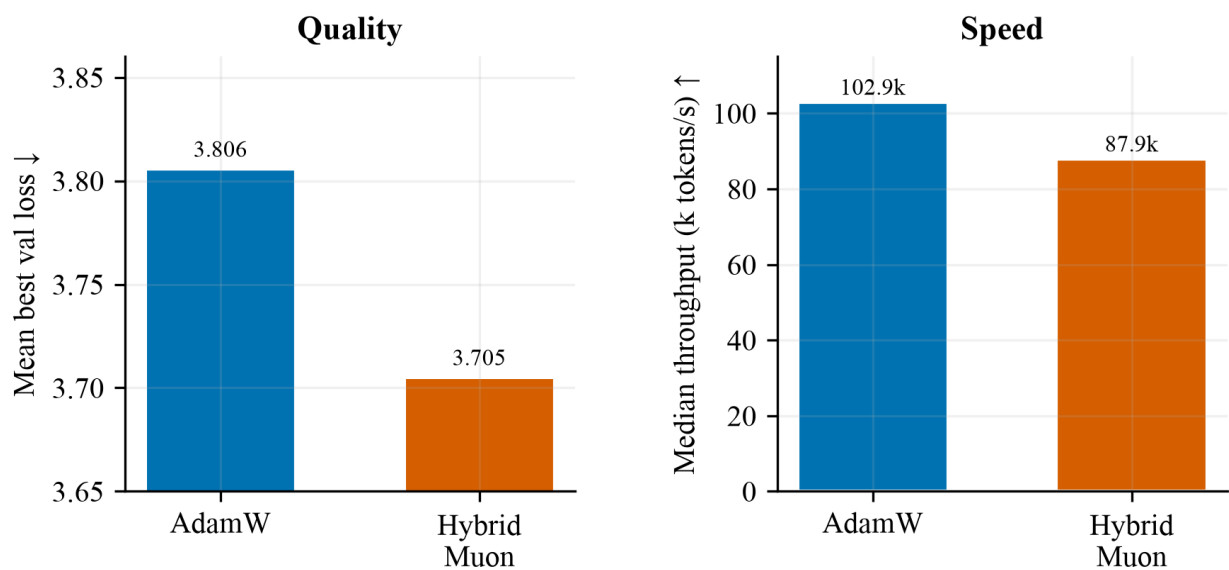


Validation-loss curves for 125M FineWeb-Edu runs across three seeds.

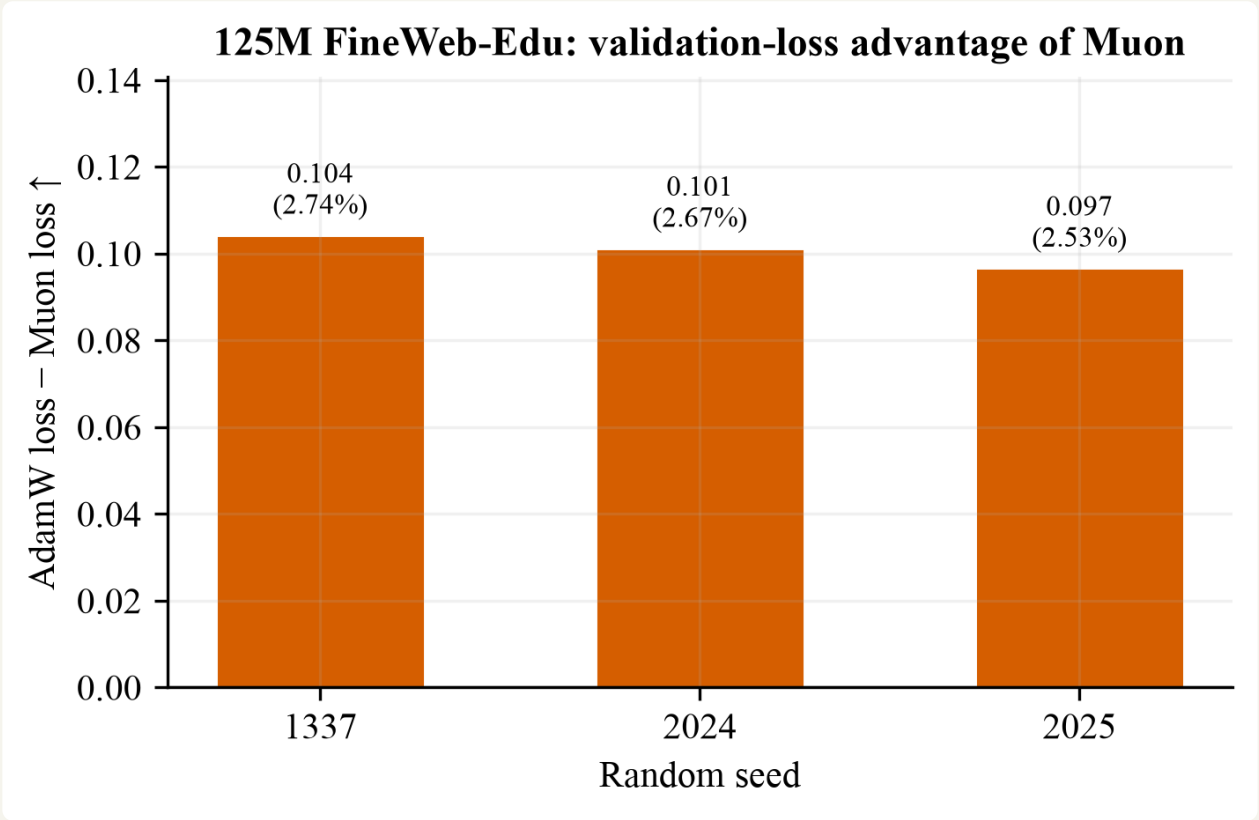
125M FineWeb-Edu aggregate

Metric	AdamW	Hybrid Muon	Delta
Mean best validation loss	3.8056 ± 0.0146	3.7049 ± 0.0181	0.1007 ± 0.0038
Mean relative improvement	—	2.65%	Muon lower
Median throughput	$\approx 102.9\text{k tokens/s}$	$\approx 87.9\text{k tokens/s}$	AdamW faster
Max allocated GPU memory	9.63 GB	9.29 GB	Similar

Tradeoff: Muon improves loss, AdamW is faster



125M tradeoff: Hybrid Muon improves mean validation loss; AdamW is faster in raw throughput.



Per-seed validation-loss advantage for Hybrid Muon on 125M FineWeb-Edu.

57M cross-dataset result

Dataset	Steps	Seeds	AdamW mean±std	Hybrid Muon mean±std	Relative improvement
TinyStories	4000	3	1.4658 ± 0.0028	1.4116 ± 0.0016	3.69%
FineWeb-Edu-500M	4000	3	3.9747 ± 0.0150	3.8645 ± 0.0144	2.77%
FineWeb-Edu-500M	8000	1	3.7377	3.6614	2.04%

Safe wording: across a narrow short-story corpus and a broader educational web-text cache, Hybrid Muon consistently achieved lower validation loss than AdamW at fixed architecture, seed, batch size, and token budget.

| Interpretation and Limits

The evidence is strong enough for a technical report, but not broad enough for a universal optimizer claim.

What the result supports

1. Hybrid Muon improved validation loss per token in our MiniGPT implementation.
2. The advantage appeared on both TinyStories and FineWeb-Edu at 57M scale.
3. The advantage persisted at 125M scale on FineWeb-Edu across three seeds.

What the result does not prove

- It does not prove that Muon is universally better than AdamW.
- It does not reproduce frontier-scale pretraining.
- It does not test every tokenizer, architecture family, context length, or dataset mixture.
- It does not claim wall-clock superiority, because AdamW had higher throughput in our runs.

Recommended next steps

The next step is not automatically a bigger model. The current evidence is already sufficient for a credible engineering report. If more GPU time is available, the best follow-up is a longer 125M FineWeb-Edu run or a 125M TinyStories run, not an immediate jump to a much larger model.

DECISION

Summarize and publish the report now. Treat larger models as future work, not a prerequisite for the current claim.

| Appendix

A. Artifact index

- `reports/57m_adamw_vs_muon_results.md`
- `reports/125m_adamw_vs_muon_3seed_results.md`
- `gpu_benchmark/downloaded_runs/compare_125m_finewebdu_3seed_4000s/`
- `figures/gen_fig_muon_benchmark.py`
- `figures/gen_3d_paper_schematics.py`

B. Glossary

Validation loss - loss measured on held-out tokens not used for gradient updates. Lower is better.

Seed - a fixed random initialization/control value that makes optimizer comparisons more reproducible.

Token budget - the number of training tokens processed by a run. In the 4000-step runs, this is 262.14M tokens.

Hybrid Muon - Muon applied to hidden 2D matrix weights, with AdamW used for fallback parameters.

C. Reproduction commands

```
python3 figures/gen_fig_muon_benchmark.py
python3 figures/gen_3d_paper_schematics.py
python3 gpu_benchmark/compare_optimizer_runs.py --help
```